

Open Eden USDO/CCIP Audit Report

Version 2.0

codehawks.cyfrin.io

March 2025

Open Eden USDO/CCIP Audit Report
Prepared by: CodeHawks

Lead Auditors
Egis Security

Disclaimer
The CodeHawks team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document.
A security audit contest does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the solidity implementation of the contracts.

Audit Dates
Mar 14th, 2025 - Mar 15th, 2025

Audit Scope
On Friday 14th March, the CodeHawks Eagles conducted an audit on the smart contracts in the USDO/CCIP repository from Open Eden, at commit hash 82c3b51d6df0884bcfcdb12bac23ad9ff369acc4.

Severity Criteria

- High: Assets can be stolen/lost/compromised directly (or indirectly if there is a valid attack path that does not have hand-wavy hypotheticals).
- Medium: Assets not at direct risk, but the function of the protocol or its availability could be impacted, or leak value with a hypothetical attack path with stated assumptions, but external requirements.
- Low: Low impact and low/medium likelihood events where assets are not at risk (or a trivia amount of assets are), state handling might be off, functions are incorrect as to natspec, issues with comments, etc.

Summary of Findings

M-01. `USDOBurnMintTokenPoolWithApproval::txnHashToTransaction` collision may result in loss of funds
M-02. User can loose value when destination is rebacing when user waits for approvals
L-01. If either bonus multipliers are 0, there will be issues on the destination chain
L-02. Check `if (balancePost < balancePre)` in `_mintIfApprovalsMet` for `USDOBurnMintTokenPoolWithApp`
L-03. Wrong `typeAndVersion` name for `USDOBurnMintTokenPoolWithApproval'
L-04. If USDO has different decimals on the different chains, it will result in wrong mint
L-05. Informational

Medium Findings

M-01. `USDOBurnMintTokenPoolWithApproval::txnHashToTransaction` collision may result in loss of funds

Summary

If user has bridged using `USDOBurnMintTokenPoolWithApproval`contract twice with same arguments, this the resulting tx hash in `releaseOrMint`will be the same and the entrie will be overriden, which results in a loss of funds.

Vulnerability Details

Nonces can be the same in `releaseOrMint` if we have > 2 token transfers with same amount in the same message, or if the message is out of order execution (the nonce for the sender will match), which will result in same `txnHash`for two different token transfers and the second one will override the first:

```Solidity
        bytes memory payload = abi.encode(
            releaseOrMintIn.remoteChainSelector,
            nonce,
            releaseOrMintIn.receiver,
            destAmount
        );
        bytes32 txnHash = keccak256(payload);
        txnHashToTransaction[txnHash] = Transaction(releaseOrMintIn.receiver, destAmount); //
@info Low -> Transaction.sender is wrong naming
        _attachLimit(destAmount, txnHash, releaseOrMintIn.remoteChainSelector);
        _authorizeTransaction(txnHash);
        _mintIfApprovalsMet(txnHash);
```

Impact

Loss of funds

Tools Used

Manual Review

Recommendations

Implement `_findUniqueHash`function, which uses a memory increasing nonce var to calculate `tnxHash`,untill `txnHashToTransaction[txnHash.sender = address(0)`

M-02. User can loose value when destination is rebacing when user waits for approvals

Vulnerability Details

In `USDOBurnMintTokenPoolWithApproval` users should wait for the bridge to approved their mint amounts. But the amount, which will be minted to user (`destAmount`) is calculated when the CCIP message is received based on `destBonusMultiplier`. The problem is that if CCIP has bridged the message right before USDO rebacing event, we safe `destAmount` that should be minted after all approvals are collected. This means that the user looses yield while waiting for that to happen and if there is a malicious approver, he may extend that period, so the damage to the victim is greater.

If there is some delay with CCIP DoN network, that may also influence the timing of execution.

Impact

Yield loss

Tools Used

Manual Review

Recommendations

Safe `srcBonusMultiplier`in `Transaction`struct and make `destAmount`calculation based on the current muliplier in `_mintIfApprovalsMet`

Low Risk Findings

L-01. If either bonus multipliers are 0, there will be issues on the destination chain

Summary

If either bonus multipliers are 0, there will be issues on the destination chain

Vulnerability Details
When calculating `destAmount` on the destination chain, 2 things can happen if the source and destination multipliers are different:

* `srcBonusMultiplier = 0` which will revert the transaction with a division by 0, effectively making the transaction unexecutable, making a loss for the user that bridged the tokens
* `destBonusMultiplier = 0` which will set `destAmount = 0` and execute the transaction, again the user will lose his funds even though the transaction would be considered successful.

Impact
Complete loss of funds for the user

Tools Used
Manual review

Recommendations
When calling `lockOrBurn` check that `IUSDO(address(i_token)).bonusMultiplier() > 0` and in `releaseOrMint` do the same.

L-02. Check `if (balancePost < balancePre)` in `_mintIfApprovalsMet` for `USDOBurnMintTokenPoolWithApp`

Summary

Check `if (balancePost < balancePre)` in `_mintIfApprovalsMet` for `USDOBurnMintTokenPoolWithApp`

Vulnerability Details

In `USDOBurnMintTokenPoolWithApproval` when `releaseOrMint` is called, it's possible the tokens will not be minted in the same transaction, instead approvers will have to call `approve` and approve the transaction.

In `releaseOrMint` there is a `balancePost < balancePre` check, which is there to prevent negative mints, but when `approve` is called and `_hasRequiredApprovals(txnHash) = true` the check isn't performed.

Impact

Code consistency and not catching a negative mint.

Tools Used

Manual review

Recommendations

Move the negative mint check inside `_mintIfApprovalsMet`

L-03. Wrong `typeAndVersion` name for `USDOBurnMintTokenPoolWithApproval`

Summary

Wrong `typeAndVersion` name for `USDOBurnMintTokenPoolWithApproval`

Vulnerability Details

`typeAndVersion` const, which is an implementation of CCIP `ITypeAndVersion` interface is the same for both `USDOBurnMintTokenPoolWithApproval` and `USDOBurnMintTokenPool` -> `"USDO BurnMintTokenPool 1.5.1"`

Impact

Duplicate names for two different contracts

Tools Used

Manual Review

Recommendations

Change `USDOBurnMintTokenPoolWithApproval` to `"USDO USDOBurnMintTokenPoolWithApproval 1.5.1"`

L-04. If USDO has different decimals on the different chains, it will result in wrong mint

Vulnerability Details

Based on [OpenEden's docs](https://docs.openeden.com/usdo/usdo-token), it is planned to expand the network support across many chain. If for example the protocol is deployed on Solana, tokens has 9 decimal, which will result in wrong token calculation, if CCIP is used.

Impact

Minting of wrong token amount (Value manipulation)

Tools Used

Manual Review

Recommendations

Consider using [TokenPool](https://github.com/smartcontractkit/chainlink/blob/31cd39a1381f9219cdb5597e1e0f248191b0b563/contracts/src/v0.8/ccip/pools/BurnMintTokenPoolAbstract.sol#L30-L45) approach for handling different token decimals

---

Informational Findings

Submitting small informational findings for code optimisation

Findings

- `Transaction.sender` -> `sender` is the token reciever. Consider renaming the var
- In `_attachLimit ` store `limit`into memory, instead of `storage`. In the function we only read from the var and storing it into memory will significantly safe gas.
- If an approval is removed, his approvals stays
    - If there is a malicios approver who has abused his role to approve and admin reacts to remove him, all the approvals stays.
    - Another problem may be if approver is removed right after user transaction has set it's limit (if the limit is high, it may not be reachable)

Mitigations
All findings were acknowledged or mitigated following this review and Egis Security conducted a mitigation check on the new commit: 9fa15174bb8d918889ee05acb676f8390f348114